



CHAPTER 3

토큰화, 품사 태깅, 불용어 제거, 텍스트 정규화

텍스트는 어떤 단계를 거쳐 처리되는가?

학습 목표

- 자연어처리 전처리의 전체 흐름 이해
 - 원시 텍스트가 수치 벡터가 되기까지의 단계별 변환 과정
- 단계별 핵심 기법 습득
 - 토큰화(Tokenization) — 단어·문장·서브워드 단위 분할
 - 품사 태깅(POS Tagging) — 토큰에 문법 범주 부여
 - 불용어 제거(Stopword Removal) — 의미 기여도가 낮은 단어 제거
 - 텍스트 정규화(Normalization) — 철자 수정, 어간 추출, 표제어 추출
 - 개체명 인식(NER), 단어 중의성 해결(WSD), 문장 경계 인식(SBD)
- 한국어 전처리의 특수성과 NLP 프로젝트 수행 절차 이해

강의 목차

- 3.0 왜 전처리가 필요한가 · 전처리 파이프라인 개요
- 3.1 토큰화 — 단어 / n-그램 / 서브워드 / 한국어 형태소
- 3.2 품사 태깅 — 태그셋, 구현, 활용
- 3.3 불용어 제거 — 개념, 구현, 주의사항
- 3.4 텍스트 정규화 — 대소문자 · 유니코드 · 정규표현식
- 3.5 철자 수정 — 편집 거리 기반 교정
- 3.6 어간 추출 / 3.7 표제어 추출
- 3.8 개체명 인식 / 3.9 단어 중의성 해결 / 3.10 문장 경계 인식
- 3.11 전처리 순서 설계와 통합 파이프라인
- 3.12 사전학습 모델 시대의 전처리 / 3.13 NLP 프로젝트 6단계

3.0 왜 전처리가 필요한가

- 원시 텍스트(raw text)는 그대로 사용할 수 없음
 - 사람이 쓴 문장은 컴퓨터에게는 단순한 문자(character)의 나열일 뿐임
 - 머신러닝 모델은 숫자만 이해하므로 텍스트를 수치로 바꿔야 함
- 원시 텍스트가 가지고 있는 문제
 - 표기 변형 : "USA", "U.S.A.", "United States" 가 같은 대상을 가리킴
 - 불필요한 정보 : HTML 태그, URL, 이모지, 반복 문자("ㅋㅋㅋㅋㅋ")
 - 철자 오류 : "Natureal", "Processin" 처럼 사전에 없는 형태
 - 형태 변화 : "study / studies / studying" 이 서로 다른 단어로 취급됨
 - 의미 기여도 차이 : "the", "is", "of" 는 의미를 거의 담지 않음
- 전처리는 이 잡음을 줄이고, 같은 의미를 같은 표현으로 모으는 작업임

3.0 전처리 파이프라인 한눈에 보기

- 전처리는 하나의 기법이 아니라, 여러 단계가 순서대로 연결된 파이프라인임
- 각 단계는 앞 단계의 출력을 입력으로 받아 점진적으로 텍스트를 정제함

NLP 텍스트 전처리 파이프라인

“컴퓨터가 이해할 수 있는 형태”로 만들기 위한 단계적 변환 과정



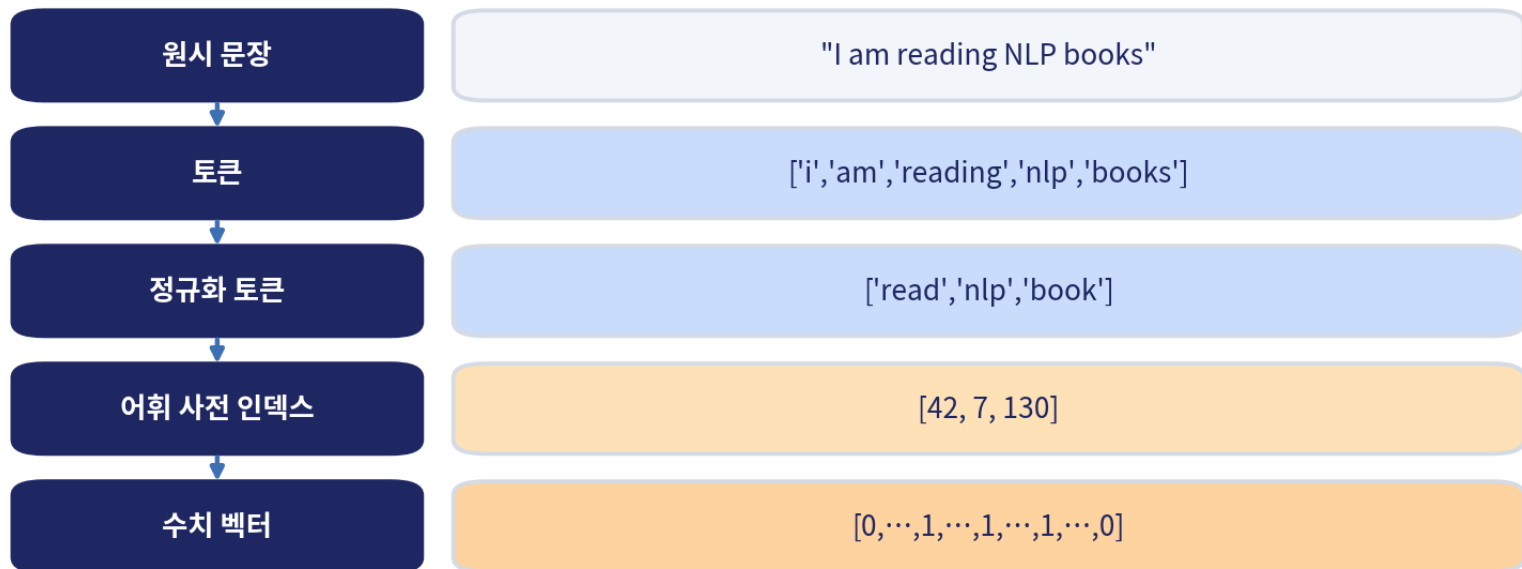
입력 예) "I am reading NLP Fundamentals!! Visit <http://ex.com>"

출력 예) ['read', 'nlp', 'fundamental'] → [12, 87, 301] → 벡터

3.0 전처리의 최종 목적

- 전처리의 목표는 “텍스트를 모델이 학습할 수 있는 수치 표현으로 바꾸는 것”임

전처리의 최종 목적 : 텍스트 → 수치 벡터



머신러닝 모델은 숫자만 이해한다 — 전처리는 그 변환을 위한 준비 작업이다

3.1 토큰화

- **토큰화 (tokenization)**

- 토큰화 (tokenization) : 텍스트를 분석의 기본 단위(토큰)로 나누는 절차. 무엇을 토큰으로 볼지는 목적에 따른 설계 결정이며 단어-문자-서브워드-형태소가 모두 토큰이 될 수 있다

- 예를 들어, 다음과 같은 문장을 생각해 보자 "I am reading a book."

- 이 문장을 토큰화하면 다음과 같은 토큰이 추출됨

```
['I', 'am', 'reading', 'a', 'book', '.']
```

- 한 번에 하나씩 토큰을 추출하며, 이러한 토큰을 유니그램(unigram)이라 함

- **토큰(token)과 타입(type)**

- 토큰 : 텍스트에 실제로 등장한 개별 단위 (중복 포함)
- 타입 : 서로 다른 토큰의 종류 (중복 제거) → 어휘 사전(vocabulary)을 구성함

3.1 토큰화 — 왜 필요한가

- 토큰화는 모든 후속 처리의 전제 조건임

- 품사 태깅, 불용어 제거, 어간 추출은 모두 "토큰 단위"로 동작함
- 어휘 사전을 만들어야 각 단어에 고유 인덱스를 부여할 수 있음
- 토큰이 잘못 나뉘면 이후 모든 단계의 결과가 함께 어긋남 (오류 전파)

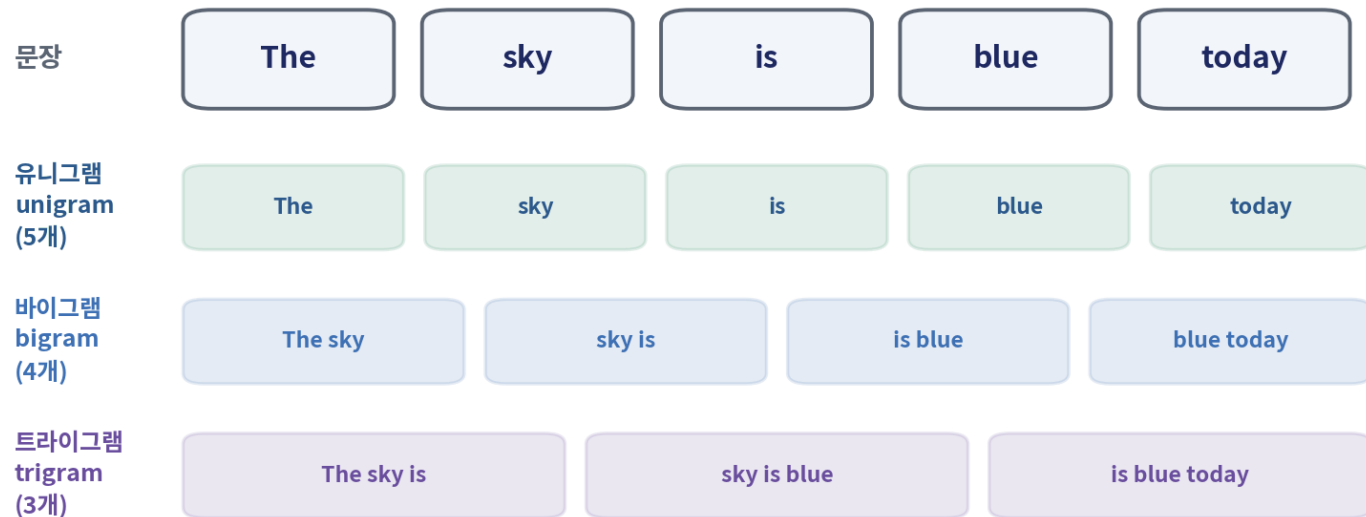
- 토큰화 단위 선택

- 단어 단위 : 직관적이지만 어휘가 커지고 OOV(사전에 없는 단어)가 발생함
- 문자 단위 : OOV는 없지만 시퀀스가 길어지고 의미 단위가 사라짐
- 서브워드 단위 : 두 방식의 절충안으로 현대 모델의 표준이 됨
- "무엇을 하나의 단위로 볼 것인가"가 토큰화 설계의 핵심 질문임

3.1 n-그램 (n-gram)

- 한 번에 두 개 또는 세 개의 토큰을 묶어 추출할 수도 있음
- 두 개는 **바이그램(bigram)**, 세 개는 **트라이그램(trigram)**이라 부름

n-그램(n-gram) : 한 번에 추출하는 연속 토큰의 개수



※ n이 커질수록 문맥 정보는 늘지만, 조합 수가 급격히 늘어 희소성(sparsity) 문제가 발생한다

3.1 n-그램의 활용과 한계

- **n-그램을 사용하는 이유**

- 단어 하나만으로는 알 수 없는 순서·문맥 정보를 담을 수 있음
- 예) "not good" 은 바이그램으로 볼 때 비로소 부정 의미가 드러남
- 예) "New York", "United States" 처럼 두 단어가 한 개체를 이루는 경우
- 활용 : 언어 모델, 검색 질의 확장, 감성 분석의 피처, 표절 탐지

- **n-그램의 한계**

- n 이 커질수록 가능한 조합 수가 지수적으로 늘어남 (희소성, sparsity)
- 학습 데이터에 없던 조합은 확률이 0이 되어 스무딩(smoothing)이 필요함
- 멀리 떨어진 단어 사이의 의존 관계는 여전히 포착하지 못함
- 실무에서는 보통 $n = 2 \sim 3$ 을 사용하며, 그 이상은 잘 쓰지 않음

3.1 토큰화가 어려운 사례

- 공백만으로 자르는 단순한 방식은 많은 경우에 잘못된 결과를 만듦

토큰화가 어려운 대표 사례

구두점

Mr. Smith went to Washington.

→ 'Mr.' 을 분리하면 의미 손상

축약형

don't / I'm / it's

→ do + n't 로 나눌 것인가?

복합 개체

United States, New York

→ 두 단어가 하나의 개체

URL·이메일

http://a.com/b?c=1

→ '/' 마다 자르면 무의미

숫자·기호

3.14, 1,000원, C++

→ 소수점·쉼표를 어떻게?

한국어 어절

학교에서를 → 학교/에서/를

→ 교착어는 어절 ≠ 단어

3.1 NLTK 토크나이저 비교

- 입력 : "Don't stop! It's 3.14 — visit a-b.com @user #NLP :)"
 - `word_tokenize` (Treebank 기반, 가장 널리 사용)
['Do', "n't", 'stop', '!', 'It', "'s", '3.14', '—', 'visit', 'a-b.com', ...]
 - `WordPunctTokenizer` (문자와 기호를 무조건 분리)
['Don', "'", 't', 'stop', '!', 'It', "'", 's', '3', '.', '14', ...]
 - `RegexpTokenizer(r"\W+")` (정규표현식으로 직접 규칙 지정)
['Don', 't', 'stop', 'It', 's', '3', '14', 'visit', 'a', 'b', 'com', 'user', 'NLP']
 - `TweetTokenizer` (해시태그·멘션·이모티콘 보존)
[..., 'a-b.com', '@user', '#NLP', ':)'] ← 멘션·해시태그·이모티콘이 한 토큰으로 보존됨
- 정답은 하나가 아니며, 데이터의 성격과 목적에 맞는 토크나이저를 선택해야 함

3.1 단순 문장의 토큰화 (실습)

- NLTK 라이브러리를 사용해 주어진 문장의 단어를 토큰화해보고자 함
 - 1. 주피터 노트북(또는 코랩)을 엮
 - 2. 새 셀을 삽입하고 다음 코드를 추가해 필요한 라이브러리를 불러옴

```
import nltk
nltk.download('punkt_tab')
from nltk import word_tokenize
```
 - 3. word_tokenize() 함수에 입력으로 문장을 전달해 토큰을 추출함

```
words = word_tokenize("I am reading NLP Fundamentals")
print(words)
```

`['I', 'am', 'reading', 'NLP', 'Fundamentals']`
 - 4. word_tokenize() 함수로 생성한 토큰 리스트를 확인할 수 있음

3.1 정규표현식 기반 토큰화 (실습)

- 규칙을 직접 정의해 원하는 형태로만 토큰을 추출할 수 있음

```
from nltk.tokenize import RegexpTokenizer
```

```
text = "Contact: hong@ex.com, 010-1234-5678 (2024-03-15)"
```

```
# 영문 단어만 추출
```

```
print(RegexpTokenizer(r"[A-Za-z]+").tokenize(text))
```

```
['Contact', 'hong', 'ex', 'com']
```

```
# 숫자를 포함한 토큰 추출
```

```
print(RegexpTokenizer(r"[0-9-]+").tokenize(text))
```

```
['010-1234-5678', '2024-03-15']
```

- 도메인 특화 텍스트(로그, 의료 기록, 법률 문서)에서 특히 유용함

3.1 문장 단위 토큰화 (실습)

- 문서를 문장 단위로 먼저 나눈 뒤 단어 토큰화를 수행하는 경우가 많음

```
from nltk import sent_tokenize, word_tokenize
```

```
text = ("NLP is fun. It is used in many fields. "
```

```
       "Dr. Kim works at A.I. Lab.")
```

```
sentences = sent_tokenize(text)
```

```
print(len(sentences))  # 4 ← A.I. 뒤에서 잘림
```

```
4
```

```
for s in sentences:
```

```
    print(word_tokenize(s))
```

```
['NLP', 'is', 'fun', '.']
```

```
['It', 'is', 'used', 'in', 'many', 'fields', '.']
```

```
['Dr.', 'Kim', 'works', 'at', 'A.I', '.']
```

```
['Lab', '.']
```

3.1 문장 단위 토큰화 (실습)

- 문서를 문장 단위로 먼저 나눈 뒤 단어 토큰화를 수행하는 경우가 많음

```
from nltk import sent_tokenize, word_tokenize
```

```
text = ("NLP is fun. It is used in many fields. "
```

```
       "Dr. Kim works at A.I. Lab.")
```

```
sentences = sent_tokenize(text)
```

```
print(len(sentences))  # 4 ← A.I. 뒤에서 잘림
```

```
4
```

```
for s in sentences:
```

```
    print(word_tokenize(s))
```

- ※ "Dr."는 punkt의 약어 사전에 있어 유지되지만 "A.I."는 사전에 없어 문장 경계로 잘못 나뉜다 → 결과가 4문장이 된다
- 도메인 약어는 직접 등록해야 한다 : `tk = PunktSentenceTokenizer(); tk._params.abbrev_types.add("a.i")`

3.1 서브워드 토큰화 — OOV 문제

- 단어 단위 토큰화는 학습 시 보지 못한 단어를 모두 [UNK]로 처리해 정보를 잃음

OOV 문제와 서브워드 토큰화

단어 단위 토큰화

"unhappiness" → [UNK]

사전에 없는 단어 = 정보 완전 손실

서브워드 토큰화

"unhappiness" → un + happi + ness

조각으로 나눠 의미 일부라도 보존

핵심 아이디어 : 자주 등장하는 문자열은 통째로,
드문 문자열은 더 작은 조각으로 분해한다

BPE · WordPiece · SentencePiece → BERT, GPT 등 현대 모델의 표준

3.1 BPE (Byte Pair Encoding)

- 가장 자주 함께 등장하는 문자 쌍을 반복적으로 하나로 병합하는 알고리즘임
- 원하는 어휘 크기에 도달할 때까지 병합을 반복함

BPE(Byte Pair Encoding) 병합 과정

초기	<code>low</w> · lower</w> · newest</w></code>	문자 단위로 분해
1회	<code>low</w> · lower</w> · newest</w></code>	(e,s) → es 빈도 최다
2회	<code>low</w> · lower</w> · newest</w></code>	(es,t) → est
3회	<code>low</w> · lower</w> · newest</w></code>	(l,o) → lo
4회	<code>low</w> · lower</w> · newest</w></code>	(lo,w) → low

3.1 서브워드 알고리즘 비교

- **BPE (Byte Pair Encoding)**
 - 빈도가 가장 높은 인접 쌍을 병합 · GPT 계열에서 사용
- **WordPiece**
 - 병합 후 언어 모델 확률(우도)이 가장 크게 증가하는 쌍을 선택
 - 단어 중간 조각에 "##" 접두어를 붙임 → BERT 계열에서 사용
- **SentencePiece**
 - 공백까지 하나의 기호(_)로 취급해 언어에 독립적으로 동작함
 - 사전 토큰화가 필요 없어 한국어·일본어·중국어에 특히 유리함
- **한국어에서는 SentencePiece(Unigram/BPE)가 사실상 표준으로 사용됨**

3.1 한국어 토큰화 — 어절 ≠ 단어

한국어 토큰화 : 어절 ≠ 단어

문장 : “나는 학교에서 책을 읽었다”

① 공백(어절) 기준 분리

['나는', '학교에서', '책을', '읽었다']

조사·어미가 붙어 어휘 폭발

② 형태소 기준 분리

['나', '는', '학교', '에서', '책', '을', '읽', '었', '다']

어간·조사·어미를 분리 → 어휘 축소

왜 이런 차이가 생기나? → 한국어는 교착어(agglutinative language)

- 하나의 어간에 조사·어미가 계속 결합한다 (학교 / 학교가 / 학교를 / 학교에서 / 학교에서만 ...)
- 띄어쓰기 규칙이 느슨하고 실제 문서에서 자주 어긋난다 (“할수있다” / “할 수 있다”)
- 같은 형태가 품사에 따라 다른 의미를 가진다 (“배”: 과일 / 신체 / 선박 / 배수)

→ 따라서 형태소 분석기(KoNLPy, Kiwi, MeCab)를 이용한 토큰화가 필요하다

3.1 한국어 토큰화의 중의성

- 띄어쓰기가 잘못되면 완전히 다른 의미로 분해될 수 있음

한국어 형태소 분석 결과 비교

입력 : “아버지가방에들어가신다”

띄어쓰기 없음(원문)	아버지가방에들어가신다	—
잘못된 분해	아버지 / 가방 / 에 / 들어가신다	“아버지 가방에 들어가신다”
올바른 분해	아버지 / 가 / 방 / 에 / 들어가 / 신다	“아버지가 방에 들어가신다”

형태소 분석기(Okt, Komoran, Mecab, Kiwi)는 문맥과 사전을 이용해 올바른 분해를 찾는다

3.1 한국어 형태소 분석 (실습)

- **KoNLPy** — 여러 형태소 분석기를 하나의 인터페이스로 제공

```
!pip install konlpy
```

```
from konlpy.tag import Okt
```

```
okt = Okt()
```

```
s = "아버지가 방에 들어가신다"
```

```
print(okt.morphs(s))
```

```
['아버지', '가', '방', '에', '들어가신다']
```

```
print(okt.pos(s))
```

```
[('아버지', 'Noun'), ('가', 'Josa'), ('방', 'Noun'), ('에', 'Josa'), ('들어가신다', 'Verb')]
```

※ KoNLPy의 Okt·Komoran·Hannanum·Kkma는 자바 (JVM)가 필요하고, Mecab-ko는 별도 바이너리 설치가 필요하며, Kiwi는 pip 설치만으로 동작함

- **Kiwi** — 자바 설치가 필요 없고 속도가 빠른 최신 분석기

```
!pip install kiwipiepy
```

```
from kiwipiepy import Kiwi
```

```
print([t.form for t in Kiwi().tokenize(s)])
```

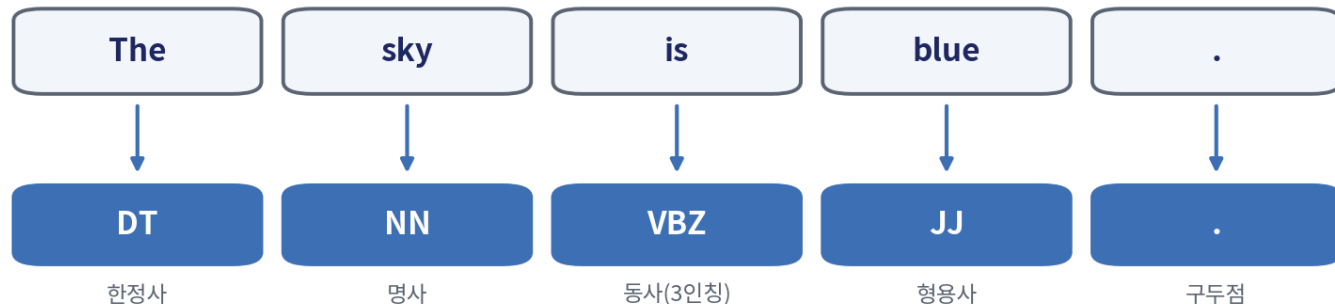
```
['아버지', '가', '방', '에', '들어가', '시', '는 다']
```

3.2 품사 태깅 (POS Tagging)

- POS(Part-of-Speech)는 품사를 의미함
 - 품사 태깅은 문장 내의 각 토큰에 해당하는 품사를 부여하는 프로세스를 의미함
 - 토큰의 품사를 알면 관심 있는 품사만 골라 분석할 수 있음

품사 태깅(POS Tagging) : 각 토큰에 품사 레이블 부여

입력 문장 : "The sky is blue."



출력 : [('The','DT'), ('sky','NN'), ('is','VBZ'), ('blue','JJ'), (',','.')]

3.2 Penn Treebank 태그셋

◦ 영어 품사 태깅에서 가장 널리 쓰이는 표준 태그 체계임 (총 36개 태그)

- NN 단수 명사 / NNS 복수 명사 / NNP 고유 명사 단수 / NNPS 고유 명사 복수
- VB 동사 원형 / VBD 과거형 / VBG 동명사·현재분사 / VBN 과거분사
- VBP 현재형(비3인칭) / VBZ 현재형(3인칭 단수)
- JJ 형용사 / JJR 비교급 / JJS 최상급
- RB 부사 / RBR 비교급 부사 / RBS 최상급 부사
- DT 한정사 / IN 전치사·종속접속사 / PRP 인칭대명사 / CC 등위접속사 / CD 기수

`nlk.help.upenn_tagset('VBZ')` # 태그의 뜻을 확인하는 방법

※ 태그를 외울 필요는 없으며, 앞 두 글자(NN/VB/JJ/RB)만 알아도 대부분 처리 가능함

3.2 품사 태깅 (실습)

- “I am reading NLP Fundamentals” 문장의 각 단어에 대한 PoS를 찾음

- 1. 필요한 라이브러리를 불러옴

```
import nltk
```

```
nltk.download('averaged_perceptron_tagger_eng')
```

```
from nltk import word_tokenize
```

- 2. 먼저 토큰을 얻기 위해 토큰화를 수행함

```
words = word_tokenize("I am reading NLP Fundamentals")
```

- 3. pos_tag() 함수를 사용해 각 단어 토큰에 대한 PoS를 찾음

```
print(nltk.pos_tag(words))
```

```
[('I', 'PRP'), ('am', 'VBP'), ('reading', 'VBG'), ('NLP', 'NNP'), ('Fundamentals', 'NNS')]
```

- PRP는 인칭대명사, VBP는 현재형 동사, VBG는 동명사·현재분사(여기서는 현재분사), NNP는 고유명사, NNS는 복수명사임

3.2 품사 태깅은 어디에 쓰이나

- ① 표제어 추출의 정확도 향상
 - lemmatize('saw') → 'saw' / lemmatize('saw', pos='v') → 'see'
- ② 특정 품사만 선택적으로 추출
 - 키워드 추출 : 명사(NN*)만 남기면 문서의 주제어를 얻을 수 있음
 - 감성 분석 : 형용사(JJ)와 부사(RB)가 감성 극성의 핵심 단서가 됨
- ③ 구문 분석 · 개체명 인식의 입력 피쳐
 - NER은 보통 품사 태깅 결과를 입력으로 받아 개체 후보를 찾음
- ④ 단어 중의성 해결의 1차 단서
 - "book"이 명사인지 동사인지에 따라 의미 후보가 달라짐

3.2 품사 태깅 방법론의 발전

- 규칙 기반 (Rule-based) *사람이 직접 규칙을 쓴다*
 - "-ly로 끝나면 부사" 같은 수작업 규칙 사용 · 언어별로 규칙 작성 부담이 큼
- 통계 기반 (HMM, CRF)
 - 앞뒤 태그의 전이 확률과 단어 출현 확률을 학습해 가장 그럴듯한 태그열을 선택
 - NLTK의 기본 태거는 평균 퍼셉트론(averaged perceptron) 기반임
- 신경망 기반 (BiLSTM-CRF, Transformer)
 - 문맥 전체를 보고 태깅하며 현재 영어 기준 97% 이상의 정확도를 보임
- 정확도가 높아도 100%는 아니므로, 태깅 오류가 후속 단계에 전파될 수 있음

3.2 한국어 품사 태깅

- 한국어는 **세종** 말뭉치 태그셋을 주로 사용함

- NNG 일반명사 / NNP 고유명사 / NNB 의존명사
- VV 동사 / VA 형용사 / VX 보조용언
- JKS 주격조사 / JKO 목적격조사 / JX 보조사
- EP 선어말어미 / EF 종결어미 / EC 연결어미
- MAG 일반부사 / MM 관형사 / SF 마침표

- 영어와의 결정적 차이

- 영어 : 단어 하나에 태그 하나 → "reading" = VBG
- 한국어 : 어절 하나가 여러 형태소로 분해되어 각각 태그를 가짐
'들어가신다' → 들어가/VV + 시/EP + ㄴ다/EF

영어보다 복잡함.

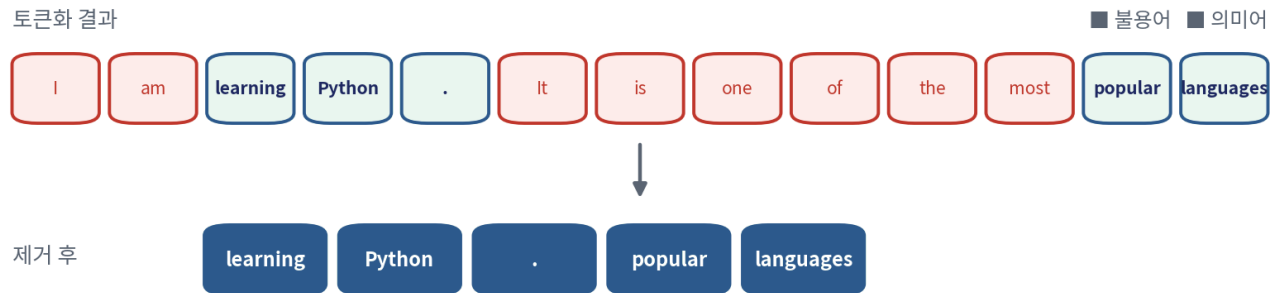
3.3 불용어 제거

- 불용어 (Stop word)

- 문장의 구성을 지원하기 위해 사용하는 일반적인 단어임
- 문장의 의미에 큰 영향을 미치지 않으므로 분석할 때는 제거하는 경우가 많음
- 불용어의 예로는 a, am, the, is, of 등이 있음

의미가 업파기 보다는
너무 흔해서 제거한다

불용어(Stop word) 제거



13개 토큰 → 5개 토큰 : 어휘 크기·연산량 감소, 핵심 의미어에 집중

※ 주의 : 감성 분석에서 “not”, “never”를 제거하면 의미가 뒤집힌다

3.3 NLTK 불용어 리스트 (실습)

- NLTK 라이브러리에서 제공하는 불용어 리스트를 사용함

- 1. 필요한 라이브러리를 불러옴

```
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords
```

- 2. words() 함수에 언어를 파라미터로 전달해 리스트를 얻음

```
stop_words = stopwords.words("english")
```

```
print(len(stop_words))
```

198

```
print(stop_words[:12])
```

['a', 'about', 'above', 'after', 'again', 'against', 'ain', 'all', 'am', 'an', 'and', 'any']

※ 축약형(you're, you've)도 목록에 포함되므로 축약형 복원 이후에 불용어를 제거해야 한다

※ stopwords.fileids() 로 지원 언어 목록을 확인할 수 있음 (한국어는 미포함)

3.3 불용어 제거 (실습)

- 주어진 문장에서 불용어를 제거해 봄

- 1. 문장을 토큰화함

```
sentence = "I am learning Python. It is one of the most popular languages"  
sentence_words = word_tokenize(sentence)
```

- 2. 각 토큰이 불용어 리스트에 포함되는지 확인하며 걸러냄

```
sentence_no_stops = " ".join(  
    [w for w in sentence_words if w.lower() not in stop_words])  
print(sentence_no_stops)
```

learning Python . one popular languages

- "am", "is", "of", "the", "most" 와 같은 불용어가 필터링 되었음을 확인할 수 있음 (단, "one"은 NLTK 기본 목록에 없어 그대로 남음)
 - **lower()를 적용하지 않으면 문장 첫 글자가 대문자인 "I", "It"이 제거되지 않음**

전역변수화

3.3 불용어 제거 시 주의사항

☆ 불용어 제거가 오히려 해로운 경우가 있음

- 감성 분석 : "not", "no", "never"를 지우면 의미가 정반대가 됨
"this is not good" → "good" (긍정으로 오분류)
- 질의응답·검색 : "to be or not to be"는 모두 불용어라 문장 자체가 사라짐
- 구문 분석 : 기능어가 사라지면 문장 구조를 복원할 수 없음
- 사전학습 모델(BERT 등) : 문맥 정보를 스스로 학습하므로 제거하지 않는 것이 일반적임

◦ 실무 권장 사항

- 기본 리스트를 그대로 쓰지 말고 도메인에 맞게 추가·제외 목록을 관리할 것

```
stop_words = set(stopwords.words("english")) - {"not", "no", "nor"}
```

```
stop_words |= {"said", "would", "also"} # 도메인 불용어 추가
```

3.3 한국어 불용어

- 한국어는 NLTK가 불용어 리스트를 제공하지 않음

- 직접 리스트를 구축하거나 품사 정보를 이용해 제거하는 방식을 사용함

- 방법 ① 직접 정의한 리스트 사용

```
ko_stop = {"의","가","이","은","들","는","좀","잘","강","과","도","를","으로","자","에",  
           "","와","한","하다"}
```

```
tokens = [t for t in tokens if t not in ko_stop]
```

- 방법 ② 품사 기준 필터링 (권장)

- 세종 태그셋(Mecab-ko-Komoran)은 조사(J*),어미(E*),기호(S*)를 접두어로 거르고, Okt는 자체 태그셋이므로 남길 품사를 지정한다

```
nouns_verbs = [w for w, p in okt.pos(s) if p in ("Noun", "Verb", "Adjective")]
```

```
세종 : [w for w, p in tagger.pos(s) if not p.startswith(("J","E","S"))]
```

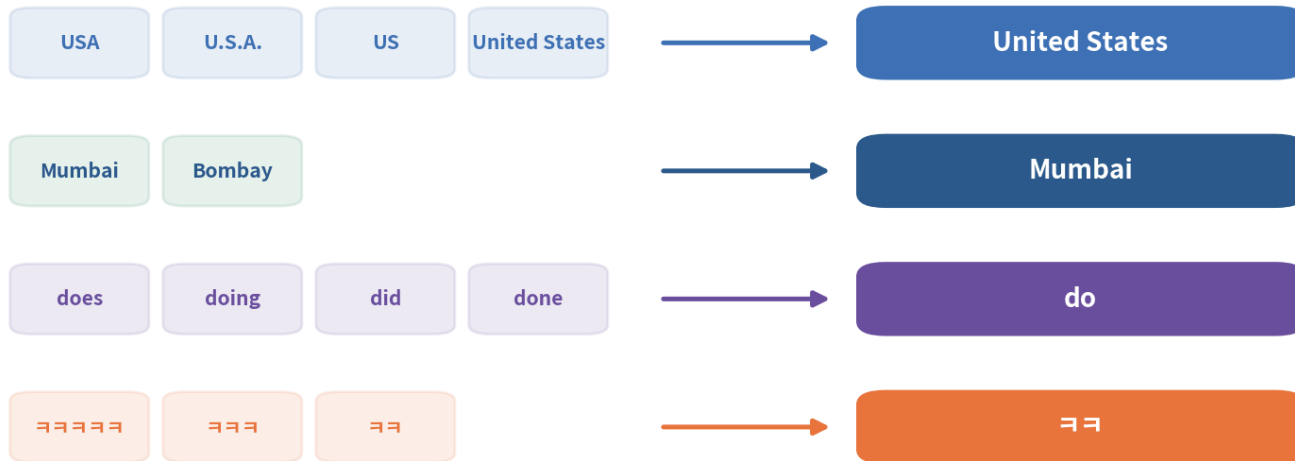
- 한국어에서는 “리스트”보다 “품사”로 거르는 편이 유지보수에 유리함

3.4 텍스트 정규화

◦ 텍스트 정규화 (text normalization)

- 다르게 표기되지만 의미적으로 동일한 표현을 하나의 표준 형식으로 바꾸는 과정임
- 예) 뭄바이와 봄베이, US와 United States, does와 doing의 기본형 do

텍스트 정규화(Text Normalization) : 표기 변형을 하나의 표준형으로



정규화 방법 : 사전/규칙 치환 · 대소문자 통일 · 유니코드 정규화 · 철자 수정 · 어간 추출 · 표제어 추출

3.4 정규화 기법 ① 대소문자와 유니코드

- 대소문자 통일 (case folding)

- "Apple", "APPLE", "apple"을 하나로 모아 어휘 크기를 줄임

```
text = text.lower()
```

- 주의 : "US(미국)"와 "us(우리)", "Apple(회사)"과 "apple(과일)"이 구분되지 않음

- 유니코드 정규화 (NFC / NFD)

- 한글은 완성형(NFC)과 조합형(NFD) 두 가지로 저장될 수 있어 겉보기에 같아도 다름

타이핑은 각기 다르게 인식
NFC로 통일 권장

```
import unicodedata
```

```
a = "한글" # NFC (완성형)
```

```
b = unicodedata.normalize("NFD", a) # NFD (조합형)
```

```
print(a == b, len(a), len(b))
```

```
False 2 6 ← 눈에는 같아 보이지만 다른 문자열
```

3.4 정규화 기법 ② 정규표현식 정제

- 실제 웹 텍스트에는 분석에 불필요한 요소가 다수 섞여 있음

```
import re
```

```
def clean(t):
```

```
    t = re.sub(r"<[^>]+>", " ", t)          # HTML 태그
```

```
    t = re.sub(r"http\S+|www\S+", " ", t)      # URL
```

```
    t = re.sub(r"\S+@\S+\S+", " ", t)         # 이메일
```

```
    t = re.sub(r"[^가-힣ㄱ-ㅎㅏ-ㅣa-zA-Z0-9ws.,!~]", " ", t) # 특수문자·이모지 (호환 자모만 보존 - NFD 조합형 자모는 삭제되므로 NFC 정규화를 먼저 할 것)
```

```
    t = re.sub(r"(\S){2,}", r"\S\S", t)        # ㅋㅋㅋㅋ → ㅋㅋ
```

```
    return re.sub(r"\S+", " ", t).strip()      # 연속 공백
```

- 정제 규칙은 반드시 순서를 고려해야 함 — 특수문자를 먼저 지우면 URL 패턴이 깨짐

3.4 규칙 기반 치환 (실습)

- `replace()` 함수로 선택한 표현을 표준형으로 바꿔 정규화한 텍스트를 생성함

```
sentence = "I visited US from UK on 22-10-18"
```

```
normalized = (sentence.replace("US", "United States")  
               .replace("UK", "United Kingdom")  
               .replace("-18", "-2018"))
```

```
print(normalized)
```

I visited United States from United Kingdom on 22-10-2018

- **규칙 기반 방식의 장단점**

- 장점 : 동작이 예측 가능하고 도메인 지식을 그대로 반영할 수 있음
- 단점 : 규칙 수가 늘면 관리가 어렵고, 부분 문자열 오치환 위험이 있음

예) `"BUS".replace("US","United States")` → `"BUnited States"`

3.5 철자 수정

◦ 철자 수정 (spelling correction)

- 오타가 있으면 같은 단어가 서로 다른 토큰으로 취급되어 어휘가 불필요하게 커짐
- 사전에 있는 단어 중 “가장 가까운” 단어로 바꿔주는 것이 기본 원리임

철자 수정의 기본 원리 : 편집 거리(Edit Distance)

“Natureal” → 사전 단어 중 편집 거리가 가장 작은 것을 선택

삽입 (insert)	cat → cart	문자 하나 추가
삭제 (delete)	cart → cat	문자 하나 제거
교체 (substitute)	cat → cut	문자 하나 바꿈
전치 (transpose)	form → from	인접 문자 자리바꿈

예) Natureal → Natural (편집 거리 1, ‘e’ 삭제)

3.5 철자 수정 (실습)

- 파이썬 `autocorrect` 라이브러리를 사용해 단어와 문장의 철자를 수정해보자

```
!pip install autocorrect
```

```
from autocorrect import Speller ; spell = Speller(lang="en")
```

- 1. 단어 단위 수정

```
print(spell("Natureal"))
```

Natural

- 2. 문장 단위 수정 — 토큰화 후 각 토큰을 수정하고 다시 결합함

```
s = word_tokenize("Ntural Luanguage teh Processin deals with insights")
```

```
corrected = " ".join([spell(w) for w in s])
```

```
print(corrected)
```

Natural Language teh Processing deals with insights

- 대부분의 오타가 수정되었으나 "teh"은 완전히 복원되지 않았음

철자수정은 완벽할수없다

3.5 철자 수정의 한계와 대안

- 편집 거리만으로는 해결되지 않는 경우

- 문맥 무시 : "I eight an apple" 의 "eight"는 철자상 정답이라 교정되지 않음
- 고유명사 오인 : 사람 이름, 브랜드명, 신조어를 오타로 판단해 잘못 고침
- 속도 : 사전 전체와 거리를 계산하므로 대용량 문서에서는 부담이 큼

- 대안

- SymSpell : 삭제 기반 인덱싱으로 수천 배 빠른 근사 검색을 제공함
- 문맥 인식 교정 : 언어 모델(BERT 등)로 앞뒤 문맥까지 고려해 교정함
- 도메인 사전 등록 : 고유명사와 전문 용어를 사전에 미리 추가함
- SNS·리뷰 데이터에서는 효과가 크지만, 정제된 문서에서는 생략해도 무방함 ☆

3.6 어간 추출

- 어간 추출 (stemming)

- 영어와 같은 언어에서 단어는 문장에 사용될 때 다양한 형태로 변형됨
- products는 굴절형, production은 파생형이다. 어간 추출은 둘을 구분하지 않고 잘라내므로 모두 product가 된다 (표제어 추출과의 결정적 차이)
- 이 단어들은 같은 의미를 가지므로 기본 형태로 변환해야 함
 - production → (어간 추출) → product
 - products → (어간 추출) → product

- 동작 방식

- 사전을 참조하지 않고 정해진 규칙으로 접미사를 잘라냄
- 따라서 매우 빠르지만, 결과가 실제 단어가 아닐 수 있음

3.6 Porter 알고리즘의 규칙 예시

- Porter Stemmer는 5단계에 걸쳐 접미사를 순차적으로 제거함

- 1a단계 : SSES → SS (caresses → caress), IES → I (ponies → poni), S → (없앰) (cats → cat)
- 1b단계 : (m>0) EED → EE (agreed → agree), (*v*)ING → (없앰) (battling → battl)
- 2단계 : ATIONAL → ATE (relational → relate), IZATION → IZE, ALISM → AL
- 3단계 : ICAL → IC (electrical → electric), FUL·NESS → (없앰) (hopeful → hope, goodness → good)
- 4단계 : AL, ANCE, ENCE, ER, IC, ABLE 등 제거
- 5단계 : 남은 E 제거 (rate → rat 방지 조건 포함)
- 규칙만 적용하므로 “battling → battl”, “was → wa” 같은 비단어가 생길 수 있음

※ 이런 한계를 보완한 것이 다음 절의 표제어 추출(lemmatization)임

3.6 어간 추출기 비교

- **PorterStemmer** (가장 널리 쓰이는 표준 알고리즘)

running→run, studies→studi, generously→gener

- **LancasterStemmer** (가장 공격적, 결과가 짧음)

running→run, studies→study, generously→gen

- **SnowballStemmer** (Porter2 — Porter의 과도 축약을 보완, 다국어 지원)

running→run, studies→studi, generously→generous

```
from nltk.stem import PorterStemmer, LancasterStemmer,
    SnowballStemmer
```

```
SnowballStemmer.languages # 지원 언어 확인 (한국어는 미지원)
```

- 과도한 축약은 서로 다른 단어를 하나로 합쳐(**over-stemming**) 의미를 훼손함

3.6 어간 추출 (실습)

- 몇 가지 단어를 어간 추출 과정에 전달해 기본 형태로 변환해 봄

- 1. 필요한 라이브러리를 불러옴

```
import nltk
```

```
stemmer = nltk.stem.PorterStemmer()
```

- 2. 단어들을 stem() 함수에 파라미터로 전달함

```
print(stemmer.stem("production"))
```

```
product
```

```
print(stemmer.stem("coming"))
```

```
come
```

```
print(stemmer.stem("firing"))
```

```
fire
```

```
print(stemmer.stem("battling"))
```

```
battl    ← 사전에 없는 형태 (어간 추출의 한계)
```

3.7 표제어 추출

- 표제어 추출 (lemmatization)

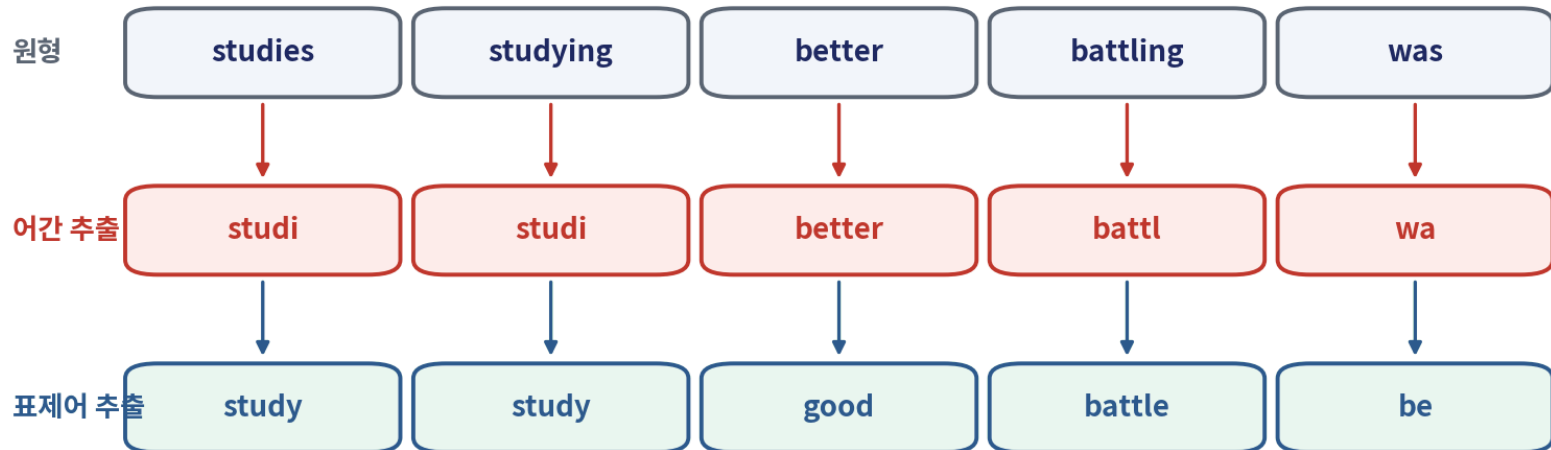
- 어간 추출은 "battling → battl" 처럼 의미 없는 결과를 만들 수 있음
- 이 문제를 극복하기 위해 사전(WordNet)을 참조해 실제 기본형을 찾는 방법임
- 단어의 품사 정보를 함께 사용하므로 훨씬 정확한 결과를 얻을 수 있음
- 다만 사전 조회와 추가 검사로 인해 어간 추출보다 처리 속도가 느림

- 표제어(lemma)란

- 사전에 표제 항목으로 올라 있는 형태 (was, is, are → be)
- 언어학적으로 타당한 기본형이므로 사람이 읽어도 자연스러움

3.7 어간 추출 vs 표제어 추출

어간 추출(Stemming) vs 표제어 추출(Lemmatization)



어간 추출 : 규칙으로 접사를 잘라냄 → 빠르지만 사전에 없는 형태 생성 가능

표제어 추출 : 사전(WordNet)·품사 정보를 이용 → 정확하지만 느림

3.7 표제어 추출 (실습)

- WordNetLemmatizer를 사용해 기본 단어를 추출해 봄

```
import nltk; nltk.download("wordnet")  
from nltk.stem.wordnet import WordNetLemmatizer  
lemmatizer = WordNetLemmatizer()
```

- 1. 기본 사용 (품사를 지정하지 않으면 명사로 간주함)

```
print(lemmatizer.lemmatize("products"))
```

product

```
print(lemmatizer.lemmatize("was"))
```

wa ← 명사로 잘못 판단

- 2. 품사를 함께 지정하면 정확도가 크게 올라감

```
print(lemmatizer.lemmatize("was", pos="v"))
```

be

- 실무에서는 pos_tag() 결과를 WordNet 품사로 변환해 함께 전달하는 것이 정석임

3.8 개체명 인식 (NER)

- 개체명 (named entity)

- 사람·기관·장소·날짜처럼 고유한 대상을 가리키는 표현을 말함
- 개체명은 일반적으로 사전에 없으므로 별도로 처리해야 함
- 주요 목표는 개체명을 식별하고 이미 정의한 범주에 매핑하는 것임

개체명 인식(NER)과 BIO 태깅 체계

토큰



■ B-XXX

개체의 시작(Begin)

■ I-XXX

개체의 내부(Inside)

■ O

개체 아님(Outside)

주요 범주 : PERSON(인명) · ORG(기관) · LOC/GPE(지명) · DATE(날짜) · MONEY(금액)

3.8 BIO 태깅과 개체 범주

◦ BIO 태깅 체계 — 여러 토큰에 걸친 개체를 표현하는 표준 방식

- B-XXX : 개체의 시작 토큰 / I-XXX : 개체의 내부 토큰 / O : 개체가 아님

New/B-LOC York/I-LOC City/I-LOC is/O big/O

◦ 대표 개체 범주

- PERSON 인명 · ORG 기관·회사 · LOC / GPE 지명·행정구역
- DATE 날짜 · TIME 시각 · MONEY 금액 · PERCENT 비율 · PRODUCT 제품명

◦ 활용 분야

- 정보 추출, 검색 질의 이해, 문서 자동 분류, 개인정보 비식별화, 지식 그래프 구축

3.8 개체명 인식 (실습)

- 문장에서 개체명을 찾아봄

```
import nltk

nltk.download("maxent_ne_chunker_tab"); nltk.download("words")

s = "We are reading a book published by Packt in New York."
tree = nltk.ne_chunk(nltk.pos_tag(word_tokenize(s)))
print([" ".join(w for w,t in st), st.label()]
      for st in tree if hasattr(st, "label")])

[('Packt', 'PERSON'), ('New York', 'GPE')]
```

- spaCy를 사용하면 더 정확하고 간결하게 처리할 수 있음

```
import spacy; nlp = spacy.load("en_core_web_sm")
print([(e.text, e.label_) for e in nlp(s).ents])

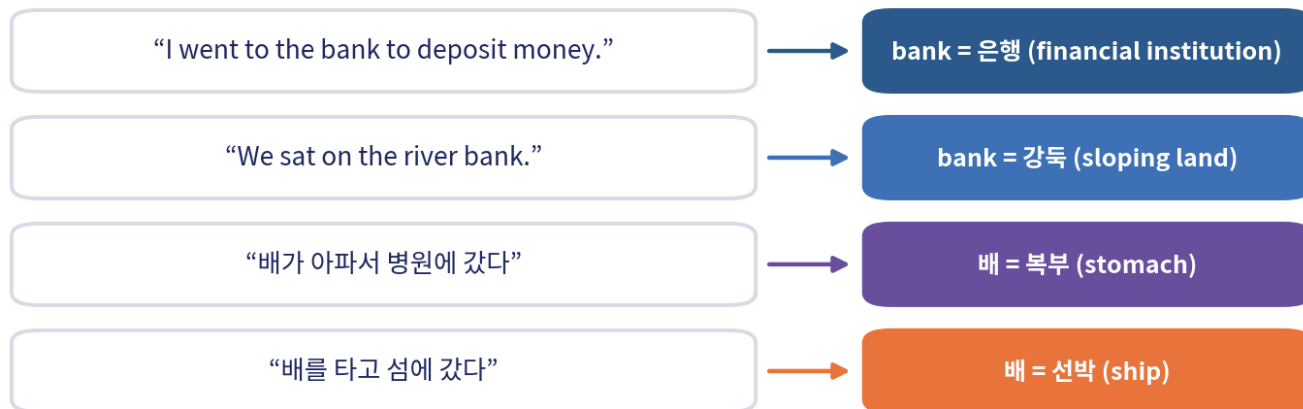
[('Packt', 'ORG'), ('New York', 'GPE')]
```

3.9 단어 중의성 해결 (WSD)

- “사귀는 친구를 보면 그 사람을 알 수 있다”
 - 이와 마찬가지로 단어의 의미도 함께 쓰인 다른 단어에 따라 달라짐
 - 철자가 같은 두 개 이상의 단어가 상황에 따라 서로 다른 의미를 가질 수 있음

단어 중의성 해결(WSD, Word Sense Disambiguation)

같은 철자, 다른 의미 — 문맥이 의미를 결정한다



대표 알고리즘 : Lesk(사전 정의문과 문맥의 어휘 중복 이용), 지도학습 분류기, 문맥 임베딩(BERT)

3.9 Lesk 알고리즘 (실습)

- 기본 아이디어

- 사전에 실린 각 의미의 정의문(gloss)과 문맥 단어의 겹침(overlap)을 세어
- 가장 많이 겹치는 의미를 정답으로 선택함

```
import nltk; nltk.download("wordnet"); nltk.download("omw-1.4")  
from nltk.wsd import lesk  
from nltk import word_tokenize
```

```
s1 = word_tokenize("I went to the bank to deposit money")  
print(lesk(s1, "bank").definition())
```

a container (usually with a slot in the top) for keeping money at home

※ 문장 1은 금융기관 의미를 놓쳤다 — 단순 Lesk는 정의문 단어 겹침만 세기 때문 (알고리즘의 한계)

```
s2 = word_tokenize("water flows beside the sloping bank")  
print(lesk(s2, "bank", "n").definition())
```

sloping land (especially the slope beside a body of water)

3.10 문장 경계 인식

- 문장 경계 인식 (Sentence Boundary Detection)
- 문서를 문장 단위로 정확히 나누는 작업임
- 요약, 번역, 질의응답은 모두 문장 단위 입력을 전제로 하므로 매우 중요함

문장 경계 인식(Sentence Boundary Detection)

“마침표 = 문장 끝”이 항상 성립하지 않는다

Dr. Smith arrived at 3 p.m.

→ Dr. / p.m. 은 문장 끝이 아님

The price is \$3.14 per unit.

→ 3.14 의 점은 소수점

Visit www.example.com now.

→ 도메인 내부의 점

He said "Stop!" Then he left.

→ 따옴표 내부의 문장 부호

해결 : 규칙 + 약어 사전 + 통계 모델 → NLTK PunktSentenceTokenizer, spaCy, Kiwi

3.10 문장 경계 인식 (실습)

- NLTK의 PunktSentenceTokenizer는 학습된 약어 사전으로 경계를 판단하므로, 사전에 없는 약어는 오분리된다

```
from nltk.tokenize import sent_tokenize
```

```
text = ("Dr. Kim joined A.I. Lab in 2020. "
```

```
        "The price is $3.14. Visit www.ex.com now!")
```

```
sents = sent_tokenize(text)
```

```
print(len(sents))
```

```
4
```

```
for i, s in enumerate(sents, 1): print(i, s)
```

```
1 Dr. Kim joined A.I.      2 Lab in 2020.
```

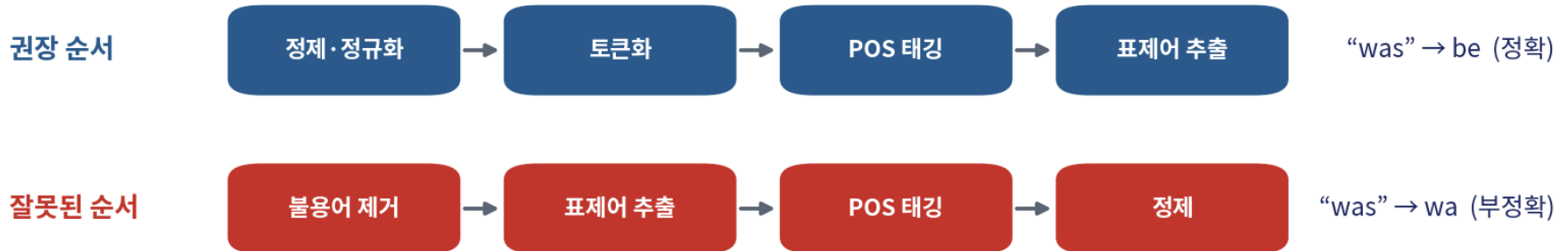
```
3 The price is $3.14.     4 Visit www.ex.com now!
```

※ \$3.14 와 www.ex.com 은 올바르게 유지되지만 미등록 약어 A.I. 는 경계로 오인된다

- 한국어는 종결어미(-다, -요, -까)를 활용하므로 Kiwi의 split_into_sents()가 유리함

3.11 전처리 순서 설계

전처리 “순서”가 결과를 바꾼다



핵심 원칙

- ① 표제어 추출은 품사 정보를 필요로 하므로 POS 태깅 뒤에 수행한다
- ② 소문자 변환을 먼저 하면 개체명 인식(NER) 성능이 크게 떨어진다
- ③ 불용어를 먼저 지우면 문맥이 사라져 POS 태깅·구문 분석이 부정확해진다
- ④ 정답은 없다 — 과제(task)와 후속 모델에 따라 순서·포함 여부를 설계한다

3.11 통합 파이프라인 (실습)

- 지금까지의 단계를 하나의 함수로 연결함

```
import unicodedata
import nltk
from nltk import word_tokenize
from nltk.corpus import stopwords, wordnet as wn
from nltk.stem import WordNetLemmatizer
```

3.11 통합 파이프라인 (실습)

- 지금까지의 단계를 하나의 함수로 연결함

```
def preprocess(text):
```

```
    text = unicodedata.normalize("NFC", text) # ① 유니코드 정규화
    text = clean(text)                        # ① 정제(clean 함수는 앞에서 정의)
    text = text.lower()                      # ② 정규화
    tokens = word_tokenize(text)             # ③ 토큰화
    tagged = nltk.pos_tag(tokens)            # ④ 품사 태깅
    tagged = [(w, t) for w, t in tagged
               if w not in stop_words and w.isalpha()] # ⑤ 불용어 제거
    return [lemmatizer.lemmatize(w, wn_pos(t))      # ⑥ 표제어 추출
            for w, t in tagged]
```

3.11 통합 파이프라인 (실습)

- 지금까지의 단계를 하나의 함수로 연결함

```
def wn_pos(t): return {"J": wn.ADJ, "V": wn.VERB, "R": wn.ADV}.get(t[0], wn.NOUN)
print(preprocess("The children were studying NLP books!"))
```

['child', 'study', 'nlp', 'book']

형용사, 동사, 명사 등 품사 지정

3.12 사전학습 모델 시대의 전처리

- BERT·GPT 이후 전처리의 역할과 관행이 크게 달라졌음

전통적 NLP 전처리 vs 사전학습 모델 시대

전통적 파이프라인

- 단어 단위 토큰화
- 불용어 제거 (필수)
- 어간·표제어 추출 (필수)
- 소문자 변환 (관행)
- 수작업 피처 설계
- BoW · TF-IDF 표현

BERT · GPT 시대

- 서브워드 토큰화 (BPE 등)
- 불용어 유지 (문맥 정보)
- 원형 복원 불필요
- 대소문자 보존 (cased 모델)
- 모델이 표현을 학습
- 문맥 임베딩 표현

그래도 전처리는 여전히 중요하다

노이즈 제거 · 인코딩 정규화 · 중복 제거 · 도메인 정제는 어떤 모델에도 반드시 필요하다

3.13 NLP 프로젝트 활성화

- 프로젝트는 여러 하위 단계로 나눌 수 있으며 이 단계들은 순차적으로 수행됨
- 각 단계를 전문화된 리소스로 수행하므로 전체 효율성이 향상되는 경향이 있음

NLP 프로젝트의 6단계



성능이 부족하면 전처리·피처·모델 단계로 되돌아가 반복 개선한다

※ 실무에서 전체 소요 시간의 60~80%는 1~2단계(데이터 수집·전처리)에 쓰인다

3.13 단계별 상세 ① 수집과 전처리

◦ 1) 데이터 수집 (Data Collection)

- 요구 사항에 따라 데이터를 모으는 단계임
- 기존 공개 데이터셋 활용, 온라인 저장소 수집, 웹 크롤링으로 자체 구축
- 예) 트윗 감성 분석 프로젝트 → 특정 키워드의 트윗을 수집함

※ 저작권·개인정보·이용약관 확인이 반드시 선행되어야 함

◦ 2) 데이터 전처리 (Data Preprocessing)

- 수집한 데이터를 정제하는 단계로 이번 장에서 배운 기법들이 모두 사용됨
- 지저분한 데이터는 모델의 효과와 정확성을 저하시킴 (Garbage In, Garbage Out)
- 예) 수집된 트윗에서 URL, 멘션, 해시태그, 이모지, 중복 트윗을 제거함

3.13 단계별 상세 ② 피처 추출과 모델 개발

◦ 3) 피처 추출 (Feature Extraction)

- 컴퓨터는 이진수만 이해하므로 텍스트를 수치 데이터로 변환해야 함
- 단어 모음(Bag of Words), TF-IDF 같은 행렬로 정제된 문서를 표현함
- 최근에는 Word2Vec, GloVe, 문맥 임베딩(BERT)을 주로 사용함

◦ 4) 모델 개발 (Model Development)

- 피처 세트가 준비되면 데이터에서 지식을 얻도록 학습할 모델을 개발함
- 통계 기반, 머신러닝 기반, 딥러닝 기반, 강화학습 기반 모델이 있음
- 예) 수치형 행렬에서 감성(긍정/부정)을 예측하는 분류 모델을 설계함

3.13 단계별 상세 ③ 평가와 배포

- **5) 모델 평가 (Model Evaluation)**

- 개발한 모델을 벤치마킹하는 과정으로, 다른 모델과 성능을 비교함
- 평가 지표 : 정확도(Accuracy), 정밀도(Precision), 재현율(Recall), F1-점수
- 클래스 불균형이 있으면 정확도만으로는 성능을 판단할 수 없음

- **6) 모델 배포 (Model Deployment)**

- 대부분의 산업에서 NLP 프로젝트의 최종 단계임
- 모델이 기존 시스템에 통합되거나, 이를 기반으로 새로운 제품을 만듦
- 예) 실시간으로 트윗에서 감성을 추출하도록 API 형태로 서비스에 배포함
- **배포 후에도 데이터 분포 변화(drift)를 모니터링하며 재학습해야 함**

요약

◦ 이번 장에서 배운 것

- NLP에 포함된 다양한 전처리 단계를 단계별로 다뤘음
 - 토큰화 → 품사 태깅 → 불용어 제거 → 정규화(철자·어간·표제어)
 - 개체명 인식, 단어 중의성 해결, 문장 경계 인식
- 전처리는 순서와 조합에 따라 결과가 달라지는 “설계”의 영역임
- 한국어는 교착어이므로 형태소 분석 기반 접근이 필요함
- 사전학습 모델 시대에도 정제·정규화의 중요성은 그대로 유지됨
- NLP 프로젝트가 거쳐야 하는 여섯 단계를 살펴보았음
- **다음 시간에는 형태소·품사부터 구문·의미까지 텍스트의 분석 계층을 차례로 올라감 (어휘 → 구문 → 의미 분석)**

참고 자료

- 전창욱, 최태균, 조중현, 신성진, 『텐서플로 2와 머신러닝으로 시작하는 자연어 처리』, 위키북스
- 홉슨 레인, 하네스 막스 하프케, 콜 하워드, 『파이썬으로 배우는 자연어 처리 인 액션』, 제이펍
- 소햄 고시(Sohom Ghosh), 드와이트 거닝, 『예제로 배우는 자연어 처리 기초』, 에이콘
- 임희석, 『자연어처리 바이블 — 핵심이론·응용시스템·딥러닝』, 휴먼싸이언스
- Jurafsky & Martin, Speech and Language Processing (3rd ed. draft)
- NLTK 공식 문서 <https://www.nltk.org>
- KoNLPy 공식 문서 <https://konlpy.org> / Kiwi <https://github.com/bab2min/kiwipiepy>

Q & A

이번 주차 내용에 대해 궁금한 점을 자유롭게 질문해 주세요